

▼ Aim - 2. Data Ingestion and Cleaning: Acquire data and prepare it for analysis.

Theory

1.What is data ingestion and why is it important in data science? Which Python library is commonly used for it?

Data ingestion refers to the process of gathering and importing data from different sources like databases, CSV files, APIs, or web services into a system for analysis. It is an essential step in data science because the accuracy of results depends heavily on the quality and availability of data. Without proper data ingestion, handling real-world data becomes difficult. In Python, the most widely used library for data ingestion and cleaning is Pandas, as it offers useful data structures like DataFrames and tools to easily load and process data from formats such as CSV, Excel, and JSON.

2.Why is data cleaning necessary before analysis or machine learning?

Data cleaning is important because real-world data is often messy and may include missing values, duplicate entries, or incorrect information. Using such data directly can lead to wrong analysis results and poor model performance. Cleaning the data ensures it is accurate, consistent, and properly formatted. This process includes handling missing values, removing duplicates, correcting errors, and standardizing formats, which ultimately improves the reliability of results.

3.How do you load a CSV file into a Pandas DataFrame?

A CSV file can be loaded into a Pandas DataFrame using the `read_csv()` function. First, the Pandas library is imported, and then the file path is passed to the function. For example, `pd.read_csv("file_name.csv")` reads the file and converts it into a DataFrame. This method is simple and efficient and allows additional options like managing headers, separators, and missing values.

4.What is the difference between `head()` and `tail()` in Pandas?

The `head()` and `tail()` functions are used to view a small portion of a dataset. The `head()` function displays the first few rows (default is five), helping to understand how the data begins. In contrast, the `tail()` function shows the last few rows, which is useful for checking the end of the dataset. Both functions are helpful when working with large datasets, as they avoid printing the entire data.

5.How can you find missing values in a dataset?

Missing values can be detected using Pandas functions like `isnull()` or `isna()`, which return a boolean result showing where data is missing. To count missing values in each column, `df.isnull().sum()` can be used. Additionally, the `info()` function provides a summary of non-missing values in each column. Identifying missing data is important so that it can be handled properly, either by removing or filling in those values.

```
import pandas as pd
import numpy as np
```

Loaded a CSV file using pandas and display first five rows.

```
from google.colab import files
uploaded = files.upload()
```

[Choose Files](#) No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun

```
df = pd.read_csv(list(uploaded.keys())[0])
```

```
df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  -
PassengerId      891 non-null    int64
Survived         891 non-null    bool
Pclass           891 non-null    int64
Name             891 non-null    object
Sex              891 non-null    object
Age              891 non-null    float64
SibSp            891 non-null    int64
Parch            891 non-null    int64
Ticket           891 non-null    object
Fare             891 non-null    float64
Cabin            147 non-null    object
Embarked         891 non-null    object
```

```

0 PassengerId 891 non-null int64
1 Survived    891 non-null int64
2 Pclass      891 non-null int64
3 Name        891 non-null object
4 Sex         891 non-null object
5 Age         714 non-null float64
6 SibSp       891 non-null int64
7 Parch       891 non-null int64
8 Ticket      891 non-null object
9 Fare        891 non-null float64
10 Cabin      204 non-null object
11 Embarked   889 non-null object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB

```

Identifying the missing value in dataset

```
df.isnull().sum()
```

	0
PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	177
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	687
Embarked	2

dtype: int64

3. Replacing the missing values with zero

```
df = df.fillna(0)
```

```
df.isnull().sum()
```

	0
PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	0
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	0
Embarked	0

dtype: int64

4. Rename column names to lower case.

```
df.columns = df.columns.str.lower()
df.head()
```

	passengerid	survived	pclass	name	sex	age	sibsp	parch	ticket	fare	cabin	embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	0	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	0	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S

5. Display Dataset shape (rows and columns)

```
print("Rows and Columns:", df.shape)
```

```
Rows and Columns: (891, 12)
```

6. Remove Duplicate rows from a dataset

```
df = df.drop_duplicates()
df.shape
```

```
(891, 12)
```

7. Handel missing values using mean or median

```
df.fillna(df.median(numeric_only=True), inplace=True)
```

8. Change Data type of Selected column

```
df['age'] = df['age'].astype(int)
df.dtypes
```

```

      0
passengerid  int64
survived      int64
pclass       int64
name         object
sex          object
age          int64
sibsp        int64
parch        int64
ticket       object
fare        float64
cabin        object
embarked     object
dtype: object

```

9. Filter rows based on a given condition

```
filtered_df = df[df['age'] > 30]
filtered_df.head()
```

passengerid	survived	pclass	name	sex	age	sibsp	parch	ticket	fare	cabin	embarked	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38	1	0	PC 17599	71.2833	C85	C
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35	0	0	373450	8.0500	0	S
6	7	0	1	McCarthy, Mr. Timothy J	male	54	0	0	17463	51.8625	E46	S

10. Save a cleaned dataset to a new CSV file

```
df.to_csv("cleaned_dataset.csv", index=False)
```

```
from google.colab import files
files.download("cleaned_dataset.csv")
```

11. Compare Missing Value Meethod

Task

Compare different missing value imputation strategies, including 0-imputation and median-imputation, analyze their impact on data distributions, clean inconsistent categorical values (e.g., 'Sex', 'Embarked'), detect and handle outliers in numerical columns (e.g., 'Age', 'Fare'), encapsulate these data cleaning steps into reusable functions, and generate a comprehensive report summarizing all data cleaning actions.

Compare Missing Value Handling Strategies

Subtask:

Apply and compare different missing value handling strategies (0-imputation and median-imputation) on the dataset, and analyze their impact on the data distribution.

Reasoning: The first step is to reload the original `titanic.csv` dataset into a new DataFrame named `df_original` to ensure we start with the original missing values, and then convert its column names to lowercase as specified in the instructions.

```
df_original = pd.read_csv(list(uploaded.keys())[0])
df_original.columns = df_original.columns.str.lower()
df_original.head()
```

passengerid	survived	pclass	name	sex	age	sibsp	parch	ticket	fare	cabin	embarked	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S

Reasoning: The previous code block failed because 'pd' was not defined. This indicates that the pandas library, which was imported in a prior cell, is not available in the current execution environment. I need to re-import pandas to resolve this `NameError` and then proceed with reloading the data and converting column names to lowercase.

```
import pandas as pd

df_original = pd.read_csv(list(uploaded.keys())[0])
df_original.columns = df_original.columns.str.lower()
df_original.head()
```

	passengerid	survived	pclass	name	sex	age	sibsp	parch	ticket	fare	cabin	embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S

Reasoning: The 'uploaded' variable from `files.upload()` is not defined across kernel restarts or different execution contexts. To fix this, I need to directly specify the filename 'titanic.csv', assuming it was uploaded and saved in the current working directory.

```
import pandas as pd

df_original = pd.read_csv('titanic.csv')
df_original.columns = df_original.columns.str.lower()
df_original.head()
```

	passengerid	survived	pclass	name	sex	age	sibsp	parch	ticket	fare	cabin	embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S

Conclusion :

In conclusion, data ingestion and data cleaning are important steps in data science that ensure data is accurate and ready for analysis. Data ingestion collects and organizes data from different sources into a structured format, while tools like Pandas make it easy to load and manage data.

Data cleaning helps fix issues such as missing values and errors in the dataset. Functions like `head()`, `tail()`, and `isnull()` assist in understanding and preparing the data. Overall, these steps improve data quality and lead to better and more reliable results.